

Guía Didáctica: Fase 4 - Modelado (Clasificación)

En esta etapa, aplicamos técnicas de Minería Predictiva para categorizar los datos.

El objetivo es construir un modelo que aprenda de los datos históricos para asignar nuevas instancias a categorías predefinidas.

1. Introducción y Objetivos

El propósito es utilizar algoritmos de aprendizaje supervisado para predecir etiquetas discretas. En el contexto de nuestro dataset, buscamos automatizar la identificación de juegos con alto impacto crítico.

2. Actividades Clave

Seleccionar Técnica de Modelado: Árbol de Decisión por su interpretabilidad y manejo de variables categóricas.

- Definir Objetivo de Negocio: Predecir "Éxito" (Meta Score >80) basado en plataforma.
- Generar Plan de Prueba: División train/test para validar generalización.

3. Herramientas Técnicas

```
import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import classification_report,
confusion_matrix

import matplotlib.pyplot as plt

import warnings
warnings.filterwarnings('ignore')

# CARGAR DATOS (ajusta ruta)
df = pd.read_csv('all_games.csv')
df_final = df[['platform', 'meta_score']].dropna().copy()
```

4. Métodos y Código

A. Variable Objetivo

```
df_final.loc[:, 'is_hit'] = (df_final['meta_score'] >
80).astype(int)
```

```
print("Distribución 'is_hit':")  
print(df_final['is_hit'].value_counts())
```

B. Features + Split

```
X = pd.get_dummies(df_final[['platform']], drop_first=True)  
y = df_final['is_hit']  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42, stratify=y  
)
```

C. Entrenamiento

```
clf = DecisionTreeClassifier(max_depth=3, random_state=42,  
    class_weight='balanced')  
clf.fit(X_train, y_train)
```

D. Evaluación

```
y_pred = clf.predict(X_test)  
print("--- Reporte de Clasificación ---")  
print(classification_report(y_test, y_pred))  
print("\n--- Matriz de Confusión ---")  
print(confusion_matrix(y_test, y_pred))
```

E. Visualización

```
plt.figure(figsize=(20,10))  
plot_tree(clf, filled=True, feature_names=list(X.columns),  
    class_names=['Bajo/Promedio', 'Éxito'],  
    fontsize=10)  
plt.title("Árbol de Decisión: Predicción de Éxitos")  
plt.show()
```